AI Dev Team — Engineering Documentation

Request
=======

implement GET /api/hello in backend/app/api/routes.py so it returns {"message":"Agent workflow works"}. Update the backend endpoint test to check the status and response JSON. Make sure both the route and its test are changed.

Implementation summary
======================

Completed:
- **Route:** Added `GET /api/hello` to `backend/app/api/routes.py` returning `{"message": "Agent workflow works"}`.
- **Test:** Created `backend/tests/test_hello.py` asserting HTTP 200 and the exact JSON response.
- **Validation:** All backend tests passed, confirming the new endpoint works and no regressions occurred.

Follow-ups:
- None. The implementation is approved and ready for merge.

Agent results
=============

planner — planned: I have enough to ground the plan. The route file is `backend/app/api/routes.py`, and the endpoint tests live in `backend/tests/` (e.g., `test_ping.py`, `test_healthcheck.py`, `test_version_endpoint.py`), with `conftest.py` providing the test client.

Final Plan

Goal: Add `GET /api/hello` returning `{"message":"Agent workflow works"}` in `backend/app/api/routes.py`, and update the backend endpoint test to assert both status code and response JSON.

Ordered tasks

1. **impl-route** — owner: *developer*
   - Edit `backend/app/api/routes.py`.
   - Add a `GET /api/hello` handler that returns the JSON body `{"message": "Agent workflow works"}` (HTTP 200).
   - Follow the existing router pattern in that file (same `APIRouter`/prefix conventions used by the sibling routes) so it registers correctly via `backend/app/main.py`.

2. **update-test** — owner: *developer*
   - Update the backend endpoint test to cover the new route.

   - Primary target: `backend/tests/test_ping.py` (the existing lightweight endpoint test). If a dedicated test is clearer, add `backend/tests/test_hello.py` instead.
   - Use the shared client from `backend/tests/conftest.py`.
   - Assert `response.status_code == 200` **and** `response.json() == {"message": "Agent workflow works"}`.
   - Ensure the route and its test are both changed (the two files above).

3. **run-tests** — owner: *tester*
   - Run the backend test suite (e.g., `pytest backend/tests`) to validate the new endpoint and confirm no regressions in existing endpoint tests (`test_ping.py`, `test_healthcheck.py`, `test_version_endpoint.py`).
   - Report pass/fail and any failures back to the developer.

Dependencies
- `update-test` depends on `impl-route` (test asserts behavior the route must provide).
- `run-tests` depends on both `impl-route` and `update-test`.

Main risks / unknowns
- **Router prefix:** `routes.py` may already define a prefix (e.g., `/api`). Need to confirm the final path resolves to exactly `/api/hello` and doesn't double-prefix.
- **Which test file to update:** `test_ping.py` is the closest match, but the request says "the backend endpoint test" — if a more specific endpoint test is intended, the developer should confirm the target file.
- **Test client setup:** `conftest.py` must expose a working app client; if it requires DB/config fixtures, the new test must reuse them to avoid setup failures.
- **Response shape:** Ensure the handler returns the exact JSON (no extra fields) so the equality assertion passes.
developer — implementation_ready: Done.

Changes:
- `backend/app/api/routes.py` — appended a `GET /api/hello` handler to the existing `/api` router that returns `{"message": "Agent workflow works"}`, with a summary/description for OpenAPI docs. No existing routes were touched.
- `backend/tests/test_hello.py` — new test that hits `/api/hello` via `TestClient` and asserts both `status_code == 200` and the exact JSON body.

Why it's safe:
- The route is additive on the existing `router` (prefix `/api`), so no changes to `main.py` or other endpoints are needed.
- The path `/api/hello` did not previously exist, so there's no duplicate-route conflict.
- The test follows the same pattern as `test_ping.py` / `test_healthcheck.py`, using the shared `app` from `app.main`.
reviewer — review_passed: The implementation is correct and complete.

Route (`backend/app/api/routes.py`):
- `GET /hello` handler appended to the existing `router = APIRouter(prefix="/api", ...)`, so the full path is `/api/hello`.
- Returns exactly `{"message": "Agent workflow works"}`.

- No existing routes were modified; no path conflicts (no prior `/hello`).
- The router is registered in `app/main.py` via `app.include_router(api_router)`, so the route is reachable.

Test (`backend/tests/test_hello.py`):
- Uses `TestClient(app)` from `app.main` (same pattern as the other endpoint tests).
- Asserts `status_code == 200` and the exact JSON body `{"message": "Agent workflow works"}`.
- Both required assertions (status + response JSON) are present.

Scope is minimal and additive; no risk to other endpoints.

VERDICT: APPROVED
tester — validated: Validation command(s) completed successfully in the backend.
docs — documented: Generated a complete workflow report from the recorded run results.

Validation
==========

python -m pytest backend/tests -q — executed
Exit code: 0

Agent documentation
==================

Source: docs/ai-dev-team-run-3a16d0e8.md
Workflow Report

Request

implement GET /api/hello in backend/app/api/routes.py so it returns {"message":"Agent workflow works"}. Update the backend endpoint test to check the status and response JSON. Make sure both the route and its test are changed.

Changed files
- backend/app/api/routes.py
- backend/tests/test_hello.py

Agent results
- developer: implementation_ready - Done.

Changes:
- `backend/app/api/routes.py` — appended a `GET /api/hello` handler to the existing `/api` router that returns `{"message": "Agent workflow works"}`, with a summary/description for OpenAPI docs. No existing routes were touched.
- `backend/tests/test_hello.py` — new test that hits `/api/hello` via `TestClient` and asserts both `status_code == 200` and the exact JSON body.

Why it's safe:
- The route is additive on the existing `router` (prefix `/api`), so no changes to `main.py` or other endpoints are needed.
- The path `/api/hello` did not previously exist, so there's no duplicate-route conflict.
- The test follows the same pattern as `test_ping.py` / `test_healthcheck.py`, using the shared `app` from `app.main`.
- reviewer: review_passed - The implementation is correct and complete.

Route (`backend/app/api/routes.py`):
- `GET /hello` handler appended to the existing `router = APIRouter(prefix="/api", ...)`, so the full path is `/api/hello`.
- Returns exactly `{"message": "Agent workflow works"}`.
- No existing routes were modified; no path conflicts (no prior `/hello`).
- The router is registered in `app/main.py` via `app.include_router(api_router)`, so the route is reachable.

Test (`backend/tests/test_hello.py`):
- Uses `TestClient(app)` from `app.main` (same pattern as the other endpoint tests).
- Asserts `status_code == 200` and the exact JSON body `{"message": "Agent workflow works"}`.
- Both required assertions (status + response JSON) are present.

Scope is minimal and additive; no risk to other endpoints.

VERDICT: APPROVED
- tester: validated - Validation command(s) completed successfully in the backend.

Changed files
==========

backend/app/api/routes.py
backend/tests/test_hello.py
docs/ai-dev-team-run-3a16d0e8.md

Workflow
========

Repository: Maheshmk18/DevAgents--AI-Software-Development-Team
Branch: agent/implement-get-api-hello-in-backend-a-3a16d0e8
Generated: 2026-10-29T03:25:55.923164+00:00